

O'REILLY LIVE TRAINING · AUTOMATA LEARNING LAB

Building Agents with OpenAI

One research task. Three runtimes.
Evidence you can inspect.

180 minutes · breaks extra · source references are clickable

What you will leave with

Build

A paper + CSV
→ a cited brief



Inspect

Tool calls, evidence,
state, and failures



Serve

Evaluation + an
authenticated app

Three ways to run the same task

Choice	Loop owner	Course state strategy
Managed Agents API	OpenAI-managed harness	Managed session
Responses	Application code	Explicit local history
Agents SDK	SDK inside your app	SDK run history

Alternatives, not mandatory migration stages.

Official runtime comparison

00 · 0:00–0:15 · 15 MINUTES

Setup & map

The model chooses actions; the runtime executes the loop; tools supply capabilities; the application enforces permissions.

Setup & map: the useful distinction

- Model → chooses a next step
- Runtime → coordinates the loop and state
- Tools → return evidence or request an action
- Application → enforces identity and permissions

Runtime comparison · Agents API concepts

COMPLETED DEMO · RUN FROM THE REPOSITORY ROOT

Put the behavior on screen

```
uv sync --extra class --extra app --extra dev
uv run python -m course preflight
uv run python -m course demo --runtime offline --follow-up --output outputs/setup
```

course/config.py

Your turn

- Draw four boxes: model, runtime, tool, application. Place choose a function, execute Python, enforce publisher permission, and keep conversation history in the correct boxes.
- Run preflight and the offline demo. Identify a primary-paper citation and explain how a CSV citation differs.

Full steps and worked solutions: [lessons/00-setup/](#)

Before we move on

Point to the model setting, the tool dispatcher, the brief schema, and the application permission check. Explain why offline success cannot verify API access.

Use the offline demo and replay. Teach the distinction between deterministic simulation and captured live output explicitly.

01 · 0:15–0:35 · 20 MINUTES

Playground: first useful agent

An answer becomes useful when we can show where it came from and what it cannot establish.

Playground: first useful agent: the useful distinction

- Task → one paper, one useful brief
- Evidence → a source and a quotation
- Missing information → say not found
- Checkpoint → explain a tool call; reject an unsupported claim

[Agents playground](#) · [Managed quickstart](#) · [Agent configuration](#)

COMPLETED DEMO · RUN FROM THE REPOSITORY ROOT

Put the behavior on screen

```
uv run python -m course demo --runtime managed --output outputs/playground-fallback
uv run python -m course demo --runtime responses --output outputs/playground-responses
uv run python -m course demo --runtime offline --output outputs/playground-offline
```

course/prompts.py

Your turn

- Use the first prompt and save the answer. Underline every factual claim and attach its supporting quotation.
- Ask the missing-information prompt. Classify each answer as supported, not found, or unsupported.

Full steps and worked solutions: [lessons/01-playground/](#)

Before we move on

Show the tool name, its input, and returned evidence. Identify a claim that goes beyond the source, even if the prose sounds confident.

Run the API-based completed demo. If no live path is available, show the offline result and ask students to audit evidence; do not describe that as a playground run.

02 · 0:35–1:05 · 30 MINUTES

Managed agent in Python

Configuration is reusable; a session is one continuing piece of work.

Managed agent in Python: the useful distinction

- Saved agent → reusable instructions and tools
- Session → turns, items, and environment
- Stream → progress and required actions
- Follow-up → reuse the session; cleanup → remove demo resources

Managed quickstart · Agent configuration · Observability and usage

COMPLETED DEMO · RUN FROM THE REPOSITORY ROOT

Put the behavior on screen

```
uv run python -m course preflight --live  
uv run python -m course demo --runtime managed --follow-up --output outputs/managed  
uv run python -m course cleanup
```

course/runtimes/managed.py

Your turn

- Trace one function request from `requires_action` to `ToolBox.dispatch` and back to `input.tool_result`. Record the two correlation IDs.
- Change the follow-up instruction to shorten the brief while retaining evidence. Explain which object must be reused.

Full steps and worked solutions: [lessons/02-managed/](#)

Before we move on

Find the session creation, stream opening, tool-result submission, completion condition, and cleanup. Explain which state the follow-up reused.

Replay a labeled capture if available. Otherwise walk through `managed.py` beside the deterministic offline two-turn demo; mark managed API access unverified.

03 · 1:05–1:30 · 25 MINUTES

Responses: expose the loop

The model asks for a function; this Python line is what actually runs it.

Responses: expose the loop: the useful distinction

- Request → the model chooses a function
- Dispatch → Python validates and executes
- Result → return the original call_id
- Repeat or finish → bound the loop; inspect the evidence

Structured outputs · Function calling

COMPLETED DEMO · RUN FROM THE REPOSITORY ROOT

Put the behavior on screen

```
uv run python -m course extract  
uv run python -m course demo --runtime responses --follow-up --output outputs/responses
```

notebooks/0.0-intro-responses-api.ipynb

Your turn

- Open `exercise.py`. Implement `matching_outputs` so every model function call receives a JSON-encoded `function_call_output` with the original `call_id`.
- Run `solution.py` locally. Predict the result of an unknown tool and a nonexistent CSV query before executing it.

Full steps and worked solutions: [lessons/03-responses/](#)

Before we move on

Point to `self.tools.dispatch` in `ResponsesRuntime`. Explain why a well-formed JSON result can contain an unsupported claim.

Run extraction offline and the local exercise solution. Replay a saved `Responses` run if present; label offline behavior as simulation.

04 · 1:30–1:55 · 25 MINUTES

Agents SDK: own the runtime

The SDK runs the loop inside our application; our application still owns its permissions and state strategy.

Agents SDK: own the runtime: the useful distinction

- Agent → shared instructions, model, output contract
- function_tool → the same application functions
- Runner → orchestration inside your app
- History → choose and own one state strategy

[Agents SDK](#) · [SDK quickstart](#) · [Runtime comparison](#)

COMPLETED DEMO · RUN FROM THE REPOSITORY ROOT

Put the behavior on screen

```
uv run python -m course demo --runtime sdk --follow-up --output outputs/sdk  
uv run python -m course demo --runtime sdk --failure read_paper --output outputs/sdk-failure
```

course/runtimes/sdk.py

Your turn

- Choose a runtime for a hosted research worker that should retain progress between tasks. State one reason and one responsibility that remains yours.
- Choose a runtime for an existing application with its own user permissions and custom database. Explain how you would retain follow-up state.

Full steps and worked solutions: [lessons/04-sdk/](#)

Before we move on

Find Agent, a function_tool, Runner.run_streamed, and the saved SDK history. Explain what the SDK replaced and what remains application code.

Inspect sdk.py alongside the offline demo; use local events or a recorded SDK output to compare state.
Do not claim Runner ran if the SDK live call was unavailable.

05 · 1:55–2:15 · 20 MINUTES

Grounding & actions

Evidence can inform the answer. It cannot grant itself permission to act.

Grounding & actions: the useful distinction

- Grounding → preserve evidence and missing information
- Preview → show the exact content
- Approval + authorization → separate decisions
- Execute once → bind to content, owner, and an idempotency record

File inputs · File search · Managed function tools

COMPLETED DEMO · RUN FROM THE REPOSITORY ROOT

Put the behavior on screen

```
uv run python -m course actions
uv run python lessons/05-grounding-actions/demo.py --runtime offline
uv run python lessons/05-grounding-actions/solution.py
```

course/actions.py

Your turn

- Open `exercise.py` and complete the authorization decision table: approval absent, permission absent, wrong owner, changed digest, valid request, repeated valid request.
- Run `demo.py` with its offline default to inspect hostile tool evidence, then choose `--runtime` responses, `sdk`, or `managed` only if that live path is prepared. Explain which application boundary prevents the retrieved instruction from publishing.

Full steps and worked solutions: [lessons/05-grounding-actions/](#)

Before we move on

Show that model text cannot publish. Explain why both approval and authorization are required, and why approving one digest cannot authorize an edited brief.

The mock action and local retrieval demos require no API. Use them to demonstrate the authorization boundary even when live model behavior is unavailable.

06 · 2:15–2:40 · 25 MINUTES

Evaluate & repair

A trace explains what happened. An evaluation judges whether it met the task.

Evaluate & repair: the useful distinction

- 10 cases → 3 normal · 2 missing · 2 failures · 2 injections · 1 denial
- Judge → evidence, tools, validity, approval
- Inspect → one failed trace and its source
- Repair → one cause; rerun the whole set

Observability and usage · Agent workflow evaluation

COMPLETED DEMO · RUN FROM THE REPOSITORY ROOT

Put the behavior on screen

```
uv run python -m course evaluate --runtime offline --output evals/results/classroom-before  
uv run python lessons/06-evaluate/solution.py
```

evals/cases.json

Your turn

- Open `exercise.py`. Create a schema-valid brief that cites an exact quotation from the supplied synthetic passage but makes a stronger claim than the quote supports.
- Run `solution.py` and explain why quote checking accepts the quote while a human reviewer should reject the claim.

Full steps and worked solutions: [lessons/06-evaluate/](#)

Before we move on

Show all ten rows, one concrete failure, the repair, and the rerun. Explain what the mechanical scores cannot establish.

Run the offline suite and inspect its recording. Label it as harness coverage; use saved live results only when their metadata identifies a genuine API run.

07 · 2:40-3:00 · 20 MINUTES

Ship & recap

The interface can request work; the authenticated backend decides whose work it is and what may happen.

Ship & recap: the useful distinction

- Backend → identity, permissions, limits, state
- UI → progress, follow-up, preview, download
- Exit → evidence + tools + continuity + failure + evaluation
- Choose → explain where your runtime and state belong

ChatKit · ChatKit advanced integration

COMPLETED DEMO · RUN FROM THE REPOSITORY ROOT

Put the behavior on screen

```
uv run python -m course access  
make backend  
make app
```

demos/research-report-app/app.py

Your turn

- Run the app as one user, produce a brief, follow up, and download it. Start a second user's session and attempt to read the first session through the backend; expect denial.
- Try publication without a publisher role, then preview and approve as an authorized user. Repeat the same approval and inspect idempotency.

Full steps and worked solutions: [lessons/07-ship/](#)

Before we move on

Demonstrate evidence, one real tool call, a follow-up turn, a handled failure, and passing evaluation results; explain your runtime choice and which checks were live.

The app defaults to offline and can demonstrate authentication, session ownership, streaming progress, downloads, and mock approval without an API call.

Five things to demonstrate

- **Evidence:** a claim and its source quotation
- **Tool use:** a real request, handler, and result
- **Continuity:** a follow-up in the same state strategy
- **Failure:** a handled outage or missing-evidence case
- **Evaluation:** fixed cases, results, and remaining limits

KEEP BUILDING

Choose the runtime. Keep the evidence.

[Course guide](#) · [instructor notes](#) · [completed demos](#) · [exercises](#)

[README.md](#) → [docs/course-guide.md](#)